LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

STACK and QUEUE

Oleh:

Farissa **NIM. 2510817120016**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Farissa
NIM : 2510817120016

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 : Implementasi Stack 6
A. Source Code 6
B. Pembahasan 7
SOAL 2 : Si Palui dan Mesin Hitung Aneh 10
A. Source Code 11
B. Output Program 12
C. Pembahasan 12
SOAL 3 : Implementasi Queue 14
A. Source Code 14
B. Pembahasan 16
SOAL 4 : Si Palui dan Deretan Angka 18
A. Source Code 19
B. Output Program 20
C. Pembahasan 20
TAUTAN GIT 22

DAFTAR GAMBAR

Gambar 1 Screenshot Hasil Jawaban Soal 2 12
Gambar 2 Screenshot Hasil Jawaban Soal 4 20

DAFTAR TABEL

Table 1 Source Code stack.h 6
Table 2 Source Code stack.cpp..... 6
Table 3 Source Code prob1.cpp 11
Table 4 Source Code queue.h..... 14
Table 5 Source Code queue.cpp 14
Table 6 Source Code prob2.cpp 19

SOAL 1 : Implementasi Stack

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 1 Source Code stack.h

```
#ifndef STACK_H
#define STACK_H

#define MAX 100

struct Stack {
    int data[MAX];
    int* top;
};

void init(Stack* s);
bool isEmpty(const Stack* s);
bool isFull(const Stack* s);
void push(Stack* s, int value);
void pop(Stack* s);
int peek(const Stack* s);

#endif
```

Table 2 Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = nullptr;
}

bool isEmpty(const Stack* s) {
    return s->top == nullptr;
}

```

```
bool isFull(const Stack* s) {
    return (s->top != nullptr && (s->top - s->data) >= MAX - 1);
}

void push(Stack* s, int value) {
    if (isFull(s)) {
        throw std::runtime_error("Stack Overflow");
    }
    if (isEmpty(s)) {
        s->top = s->data;
    } else {
        s->top++;
    }
    *(s->top) = value;
}

void pop(Stack* s) {
    if (isEmpty(s)) {
        throw std::runtime_error("Stack Underflow");
    }
    if (s->top == s->data) {
        s->top = nullptr;
    } else {
        s->top--;
    }
}

int peek(const Stack* s) {
    if (isEmpty(s)) {
        throw std::runtime_error("Stack is Empty");
    }
    return *(s->top);
}
```

B. Pembahasan

- Source Code stack.h:

File stack.h berfungsi sebagai fondasi atau kontrak definisi yang menentukan bagaimana struktur data tumpukan akan dibentuk dan diakses oleh bagian program lainnya. Karakter pertama yang muncul adalah *header guards* berupa #ifndef STACK_H, #define STACK_H, dan #endif; karakter ini sangat krusial dalam pemrograman C++ untuk memastikan bahwa isi file ini tidak didefinisikan ulang secara berulang kali oleh *compiler*, yang dapat menyebabkan eror fatal. Selanjutnya, terdapat karakter makro #define MAX 100 yang

menetapkan batas kapasitas statis bagi tumpukan, memberikan kepastian memori sebesar 100 slot integer agar program berjalan dengan stabil dan terukur.

Inti dari file ini terletak pada karakter struct Stack, sebuah tipe data bentukan yang menggabungkan dua komponen utama dalam satu kesatuan: array int data[MAX] sebagai wadah penyimpanan fisik elemen-elemen angka, dan int* top yang merupakan sebuah karakter *pointer* integer. Penggunaan *pointer* top di sini adalah elemen kunci yang membedakan implementasi ini, karena ia bertugas menunjuk langsung ke alamat memori elemen teratas, bukan sekadar menyimpan indeks angka biasa. Terakhir, file ini mendeklarasikan prototipe atau karakter antarmuka fungsi mulai dari init, isEmpty, isFull, hingga fungsi operasional push, pop, dan peek. Karakter const pada parameter beberapa fungsi pengecekan menunjukkan komitmen desain terhadap keamanan data, memastikan bahwa fungsi tersebut hanya bersifat membaca (*read-only*) dan tidak akan secara tidak sengaja mengubah isi tumpukan saat melakukan pengecekan status.

- Source Code stack.cpp:

Program ini diawali dengan karakter direktif #include "stack.h" untuk menghubungkan logika fungsi dengan definisi struktur data yang sudah dibuat, serta #include <stdexcept> yang menyediakan karakter sistem untuk menangani kesalahan saat program berjalan. Fungsi pertama, void init(Stack* s), memiliki peran vital sebagai inisiator yang mengatur karakter *pointer* s->top ke nilai nullptr, sebuah langkah awal untuk memastikan tumpukan benar-benar dalam keadaan kosong secara logis sebelum digunakan. Selanjutnya, terdapat karakter fungsi pengecekan isEmpty yang mengembalikan nilai kebenaran berdasarkan status nullptr pada top, serta fungsi isFull yang menggunakan karakter logika matematika kompleks (s->top - s->data) >= MAX - 1. Operasi pengurangan *pointer* ini secara cerdas menghitung jarak antara alamat memori elemen teratas dengan alamat awal array untuk mendeteksi apakah kapasitas maksimal telah tercapai.

Proses utama manipulasi data dilakukan melalui fungsi push dan pop yang sarat dengan karakter pengaman. Di dalam fungsi push, terdapat instruksi kondisional if (isFull(s)) yang akan memicu karakter throw std::runtime_error("Stack Overflow") jika tumpukan sudah penuh, sehingga mencegah program menulis data ke memori yang tidak sah. Karakter operasional s->top++ digunakan untuk menaikkan posisi *pointer* ke slot memori berikutnya sebelum nilai baru dimasukkan menggunakan karakter dereferensi *(s->top) = value. Sebaliknya, fungsi pop bertugas menurunkan posisi *pointer* top melalui karakter s->top--, namun sebelumnya ia akan memeriksa kondisi kekosongan tumpukan untuk memicu pesan eror "Stack Underflow" jika diperlukan. Terakhir, fungsi int peek bertindak sebagai karakter

pengakses data yang bersifat *read-only*; ia hanya mengambil nilai yang ditunjuk oleh top menggunakan operator dereferensi tanpa mengubah posisi *pointer* sedikit pun, memastikan integritas urutan tumpukan tetap terjaga bagi pengguna program.

SOAL 2 : Si Palui dan Mesin Hitung Aneh

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

• sebuah bilangan bulat, atau

• sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

• Mesin membaca simbol dari kiri ke kanan.

• Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.

• Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir. Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

• $1 \leq n \leq 100$

• $-1000 \leq Ai \leq 1000$

Keterangan:

• n adalah jumlah simbol dalam ekspresi

• Ai adalah nilai bilangan pada input • Simbol yang valid hanya: ‣ bilangan bulat ‣ operator: +, -, *, /

Format Input

N

A1 A2 ... An

Keterangan:

• Baris pertama n menyatakan jumlah atau panjang symbol

• Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	**Output**
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

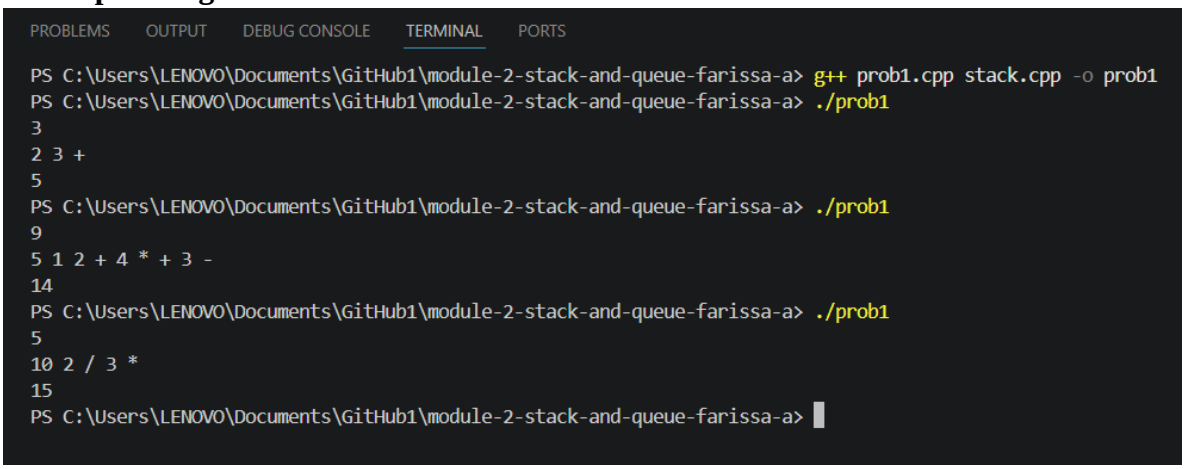
A. Source Code

Table 3 Source Code prob1.cpp

```
#include <iostream>
#include <string>
#include "stack.h"

int main() {
    int n;
    if (!(std::cin >> n)) return 0;

    Stack s;
    init(&s);

    for (int i = 0; i < n; ++i) {
        std::string token;
        std::cin >> token;

        if (isdigit(token[0]) || (token.length() > 1 && token[0] == '-')) {
            push(&s, std::stoi(token));
        } else {
            int val2 = peek(&s); pop(&s);
            int val1 = peek(&s); pop(&s);

            if (token == "+") push(&s, val1 + val2);
            else if (token == "-") push(&s, val1 - val2);
            else if (token == "*") push(&s, val1 * val2);
```

```
            else if (token == "/") push(&s, val1 / val2);
        }
    }
    std::cout << peek(&s) << std::endl;
    return 0;
}
```

B. Output Program



Gambar 1 Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Program ini diawali dengan karakter direktif #include <iostream> yang berfungsi menyediakan alat untuk komunikasi dasar melalui terminal, #include <string> yang memungkinkan program mengenali dan mengolah teks kompleks, serta #include "stack.h" yang bertindak sebagai jembatan untuk mengakses struktur data tumpukan eksternal yang telah didefinisikan sebelumnya. Di dalam fungsi utama int main(), terdapat karakter variabel n bertipe integer yang bertugas menangkap jumlah total simbol yang akan diinputkan oleh pengguna, sementara instruksi if (!(std::cin >> n)) berfungsi sebagai pengaman untuk segera mengakhiri program jika input yang dimasukkan tidak valid. Karakter utama dalam pengelolaan data adalah Stack s, sebuah objek struktur yang segera disiapkan kondisinya melalui perintah init(&s) agar *pointer* internalnya berada pada posisi awal yang benar sebelum proses hitung dimulai.

Logika pemrosesan simbol dikendalikan oleh karakter perulangan for (int i = 0; i < n; ++i), yang menjamin setiap bagian input diperiksa tepat satu kali dari kiri ke kanan. Di dalam perulangan ini, setiap simbol dibaca ke dalam variabel token yang bertipe string agar bisa menampung angka maupun operator karakter tunggal. Karakter logika kondisional if

(isdigit(token[0]) || (token.length() > 1 && token[0] == '-')) memainkan peran krusial sebagai penyaring; ia tidak hanya mengenali angka positif melalui fungsi isdigit, tetapi juga mampu mengidentifikasi angka negatif dengan memeriksa keberadaan tanda minus di awal teks yang panjangnya lebih dari satu karakter. Jika penyaring ini mendeteksi angka, instruksi std::stoi(token) akan mengubah teks tersebut menjadi bilangan bulat sesungguhnya sebelum dimasukkan ke dalam tumpukan melalui perintah push.

Sebaliknya, jika token yang terbaca adalah operator, program akan menjalankan karakter operasional pengambilan data, yaitu int val2 = peek(&s); pop(&s); dan int val1 = peek(&s); pop(&s);. Di sini, urutan pengambilan sangat penting karena nilai yang terakhir masuk ke tumpukan diambil terlebih dahulu sebagai operan kedua (val2) untuk menjaga kebenaran matematis pada operasi pengurangan dan pembagian. Setelah dua angka tersebut diamankan, karakter kontrol if-else if akan mencocokkan token dengan empat operator standar: + untuk penjumlahan, - untuk pengurangan, * untuk perkalian, dan / untuk pembagian integer. Hasil dari perhitungan antara val1 dan val2 tersebut langsung dikembalikan ke dalam tumpukan menggunakan push agar bisa digunakan untuk operasi selanjutnya. Sebagai penutup, setelah semua iterasi selesai, karakter std::cout << peek(&s) akan memanggil nilai terakhir yang tersisa di puncak tumpukan dan menampilkannya ke layar, diikuti dengan perintah return 0 yang menandakan bahwa seluruh rangkaian proses mesin hitung telah berjalan dengan sukses dan tanpa kendala.

SOAL 3 : Implementasi Queue

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 4 Source Code queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#define MAX 100

struct Queue {
    int data[MAX];
    int* front;
    int* rear;
};

void init(Queue* q);
bool isEmpty(const Queue* q);
bool isFull(const Queue* q);
void enqueue(Queue* q, int value);
void dequeue(Queue* q);
int front(const Queue* q);
int back(const Queue* q);

#endif
```

Table 5 Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q){
    q->front = q->data;
```

```
    q->rear = q->data - 1;
}

bool isEmpty(const Queue* q){
    return q->front == q->data && q->rear == q->data - 1;
}

bool isFull(const Queue* q){
    if (isEmpty(q)) return false;

    const int* nextRear;

    if (q->rear == q->data + MAX - 1)
        nextRear = q->data;
    else
        nextRear = q->rear + 1;
    return nextRear == q->front;
}

void enqueue(Queue* q, int value){
    if (isFull(q))
        throw std::overflow_error("Queue penuh!");

    if (isEmpty(q)){
        q->front = q->data;
        q->rear = q->data;
    } else {
        if (q->rear == q->data + MAX - 1)
            q->rear = q->data;
        else
            q->rear++;
    }
    *(q->rear) = value;
}

void dequeue(Queue* q) {
    if (isEmpty(q))
        throw std::underflow_error("Queue kosong!");

    if (q->front == q->rear){
        q->front = q->data;
        q->rear = q->data - 1;
    } else {
        if (q->front == q->data + MAX - 1)
            q->front = q->data;
        else
```

```
            q->front++;
    }
}

int front(const Queue* q){
    if (isEmpty(q))
        throw std::underflow_error("Queue kosong!");

    return *(q->front);
}

int back(const Queue* q){
    if (isEmpty(q))
        throw std::underflow_error("Queue kosong!");

    return *(q->rear);
```

B. Pembahasan

- Source Code queue.h

Program ini diawali dengan karakter *header guards* yaitu #ifndef QUEUE_H, #define QUEUE_H, dan diakhiri dengan #endif. Karakter-karakter ini berfungsi sebagai pengaman kompilasi yang mencegah terjadinya pendefinisian ulang struktur data jika file ini dipanggil oleh lebih dari satu file .cpp, sehingga menghindari eror *duplicate definition*. Selanjutnya, terdapat karakter makro #define MAX 100 yang secara konstan menetapkan batas kapasitas fisik antrean sebesar 100 elemen integer, memastikan alokasi memori yang terukur dan efisien.

Komponen utama dari code ini adalah karakter struct Queue, sebuah tipe data bentukan yang mengelompokkan tiga elemen esensial: array int data[MAX] sebagai media penyimpanan utama, serta dua karakter *pointer* integer yaitu int* front dan int* rear. Berbeda dengan implementasi indeks biasa, penggunaan *pointer* di sini memungkinkan program untuk menunjuk langsung ke alamat memori elemen pertama dan elemen terakhir dalam antrean, yang memberikan fleksibilitas lebih dalam manipulasi data tingkat rendah. Code ini juga mendeklarasikan karakter antarmuka fungsi-fungsi operasional seperti init, isEmpty, isFull, enqueue, dequeue, serta fungsi pengakses front dan back. Penggunaan karakter const pada parameter fungsi pengecekan status (seperti pada isEmpty dan isFull) merupakan praktik pemrograman yang baik untuk menjamin bahwa fungsi tersebut hanya akan membaca data tanpa mengubah alamat *pointer* sedikit pun, sehingga integritas data antrean tetap terjaga.

- Source Code queue.cpp

Implementasi antrean pada code ini menggunakan konsep Circular Queue berbasis *pointer* untuk mengoptimalkan penggunaan ruang memori array statis. Program diawali dengan karakter direktif #include "queue.h" dan #include <stdexcept> untuk mendukung manajemen struktur data dan penanganan kesalahan. Karakter fungsi void init(Queue* q) memiliki logika yang unik, di mana ia mengatur karakter *pointer* q->front ke alamat awal array, namun mengatur q->rear ke satu posisi sebelum awal array (q->data - 1). Hal ini dilakukan agar karakter fungsi isEmpty dapat dengan mudah mengenali kondisi kosong melalui perbandingan posisi kedua *pointer* tersebut terhadap titik awal data.

Karakter fungsi isFull pada kode ini jauh lebih cerdas karena menggunakan karakter variabel bantuan const int* nextRear untuk memprediksi posisi berikutnya. Ia mengecek apakah posisi setelah rear akan menabrak front, baik melalui pergeseran linear maupun saat harus berputar kembali dari ujung array (q->data + MAX - 1) ke awal array. Logika perputaran ini juga diterapkan pada karakter fungsi enqueue dan dequeue. Saat melakukan enqueue, jika rear sudah berada di ujung maksimal array, ia tidak akan berhenti melainkan diatur kembali ke posisi q->data agar memori yang kosong di bagian depan bisa digunakan kembali. Sebaliknya, pada fungsi dequeue, karakter *pointer* q->front akan bergeser maju atau berputar kembali ke awal untuk menghapus akses ke elemen terlama secara sirkular.

Karakter fungsi front dan back bertindak sebagai pengakses data yang dilengkapi dengan karakter pengaman berupa std::underflow_error. Jika antrean terdeteksi kosong, program akan melemparkan pesan kesalahan untuk mencegah akses memori ilegal. Namun jika tersedia, fungsi ini akan mengembalikan nilai yang ditunjuk oleh *pointer* masing-masing melalui karakter dereferensi *(q->front) dan *(q->rear). Pendekatan sirkular ini memastikan bahwa antrean tidak akan pernah "penuh semu" selama masih ada slot kosong di dalam array, menjadikannya implementasi yang jauh lebih efisien dibandingkan antrean linear biasa.

SOAL 4 : Si Palui dan Deretan Angka

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

• $1 \leq n \leq 100$

• $-1 \leq k \leq n$

• $-1000 \leq Ai \leq 1000$

Keterangan:

• n: jumlah elemen

• k: ukuran jendela

• Ai: elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

• Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk

• Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	**Output**
5 3 1 2 3 4 5	6 9 12

5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Table 6 Source Code prob2.cpp

```
#include <iostream>
#include "queue.h"

int main() {
    int n, k;
    if (!(std::cin >> n >> k)) return 0;

    Queue q;
    init(&q);

    int current_sum = 0;
    for (int i = 0; i < n; i++) {
        int val;
        std::cin >> val;

        enqueue(&q, val);
        current_sum += val;

        if (i >= k) {
            current_sum -= front(&q);
            dequeue(&q);
        }

        if (i >= k - 1) {
            std::cout << current_sum << (i == n - 1 ? "" : " ");
        }
    }
    std::cout << std::endl;

    return 0;
}
```

B. Output Program

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\LENOVO\Documents\GitHub1\module-2-stack-and-queue-farissa-a> g++ prob2.cpp queue.cpp -o prob2
PS C:\Users\LENOVO\Documents\GitHub1\module-2-stack-and-queue-farissa-a> ./prob2
5 3
1 2 3 4 5
6 9 12
PS C:\Users\LENOVO\Documents\GitHub1\module-2-stack-and-queue-farissa-a> ./prob2
5 5
1 2 3 4 5
15
PS C:\Users\LENOVO\Documents\GitHub1\module-2-stack-and-queue-farissa-a> ./prob2
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
PS C:\Users\LENOVO\Documents\GitHub1\module-2-stack-and-queue-farissa-a>
```

Gambar 2 Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Program ini diawali dengan karakter direktif #include <iostream> untuk menangani input-output data dan #include "queue.h" sebagai karakter penghubung untuk menggunakan fungsi antrean yang telah dibuat sebelumnya. Di dalam fungsi utama int main(), terdapat karakter variabel n dan k yang masing-masing bertugas menyimpan total jumlah elemen dan ukuran jendela pengamatan. Instruksi if (!(std::cin >> n >> k)) berfungsi sebagai karakter pengaman untuk memastikan program berhenti jika input tidak sesuai. Karakter utama dalam pengelolaan data adalah objek Queue q yang segera disiapkan melalui perintah init(&s), serta variabel current_sum yang diinisialisasi dengan nilai 0 untuk menyimpan akumulasi nilai angka di dalam jendela aktif.

Logika pemrosesan data dikendalikan oleh karakter perulangan for (int i = 0; i < n; i++) yang membaca setiap angka satu per satu ke dalam variabel val. Di dalam loop ini, setiap angka yang masuk segera diproses oleh karakter fungsi enqueue(&q, val) untuk dimasukkan ke dalam antrean, sementara karakter operasional current_sum += val menambahkan nilai tersebut ke total hitungan saat ini. Karakter logika kondisional if (i >= k) memainkan peran krusial; ketika jumlah elemen yang masuk sudah melampaui ukuran jendela k, program akan menjalankan instruksi current_sum -= front(&q) untuk mengurangi nilai elemen tertua dan diikuti oleh dequeue(&q) untuk mengeluarkannya dari antrean. Terakhir, karakter kondisional if (i >= k - 1) memastikan bahwa hasil jumlah hanya akan ditampilkan melalui std::cout ketika jendela sudah terisi penuh sebanyak k elemen, dengan tambahan karakter

ternary operator (i == n - 1 ? "" : " ") untuk mengatur spasi antar hasil agar sesuai dengan format *output* yang diminta.

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-farissa-a.git